

Obscura Messenger – Real-time Chat App Testing

Ева Смилеска 221053

GitHub репозиториум:

- Изворен код: https://github.com/smileska/obscura_v2

1. Преглед на проектот

Obscura Messenger е апликација за комуникација во реално време изградена со Django и Django Channels. Апликацијата содржи:

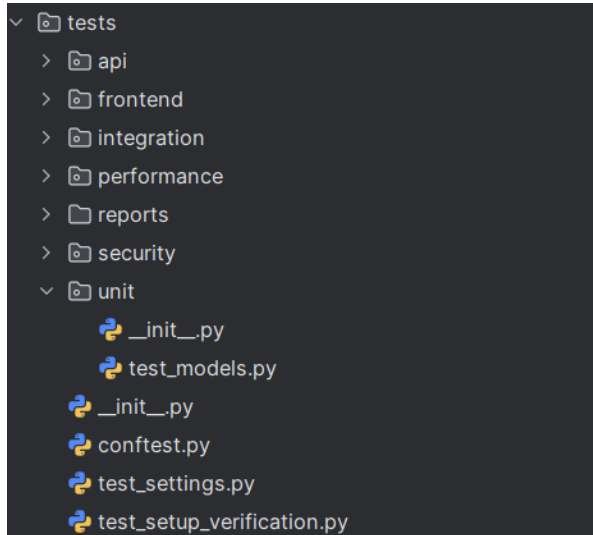
- Размена на пораки во реално време со WebSocket поддршка
- Приватни разговори помеѓу корисници
- Групни соби за разговор со административни контроли
- Реакции на пораки и споделување слики
- Автентикација на корисници и управување со профили
- Респонзивен дизајн со Bootstrap 5

Техноллогии

- **Backend:** Django 5.1.7 со Django Channels 4.2.0
- **База на податоци:** PostgreSQL 15
- **Cache/Broker:** Redis 7
- **Frontend:** Django Templates, Bootstrap 5, JavaScript
- **Deployment:** Docker, Kubernetes
- **Тестирање:** pytest екосистем со 15+ специјализирани алатки

2. Архитектура на тестирање

Архитектурата на тестирање следи повеќеслоен пристап со сеопфатна покриеност на сите слоеви од апликацијата:



Принципи за организација на тестови

1. **Разделување на одговорности:** Секоја категорија тестови се фокусира на специфични аспекти
2. **Изолација:** Тестовите се извршуваат независно без странични ефекти
3. **Повторливост:** Конзистентни резултати низ различни средини
4. **Скалабилност:** Лесно додавање нови тест случаи
5. **Одржливост:** Јасна структура и документација

3. Алатки и техники за тестирање

Основен framework за тестирање

pytest (v7.4.3) - Примарен framework за тестирање

- **Предности:** Моќни fixtures, параметризација, опширен plugin екосистем
- **Употреба:** Сите откривања на тестови, извршување и известување
- **Конфигурација:** Прилагоден pytest.ini со маркери и оптимизации

pytest-django (v4.7.0) - Django интеграција

- **Цел:** Управување со база на податоци, Django подесувања
- **Карактеристики:** Transaction rollback, fixture управување, сетирање overrides

Напредни алатки за тестирање

Locust (v2.17.0) - Тестирање на оптоварување и перформанси

- **Цел:** Симулирање на истовремени корисници и мерење системски перформанси
- **Карактеристики:** Web-based UI, дистрибуирано тестирање, прилагодени кориснички однесувања
- **Сценарија:** Тестирање оптоварување на chat, stress тестирање на API endpoints

Factory Boy (v3.3.0) - Генерирање тест податоци

- **Цел:** Создавање реалистични тест податочни објекти
- **Бенефити:** Конзистентно, одржливо создавање тест податоци

Faker (v20.1.0) – Генерирање на синтетички податоци

- **Употреба:** Генерирање реалистични usernames, emails, пораки
- **Интеграција:** Комбинирано со Factory Boy за сеопфатно создавање на податоци

pytest-cov (v4.1.0) - Анализа на покриеност на код

- **Цел:** Мерење на покриеност на тестови и идентификување нетестиран код
- **Излез:** HTML извештаи со визуелизација на покриеност линија по линија

pytest-xdist (v3.3.1) - Паралелно извршување тестови

- **Цел:** Извршување тестови паралелно за намалување време на извршување
- **Конфигурација:** Оптимизирано за test suites со интензивна употреба на база

Bandit (v1.7.5) - Скенирање безбедносни ранливости

- **Цел:** Статичка анализа за вообичаени безбедносни проблеми
- **Интеграција:** Автоматизирано скенирање на Python код за ранливости

Safety (v2.3.4) – Проверка на ранливостите на зависности

- **Цел:** Скенирање зависности за познати безбедносни ранливости
- **Употреба:** Континуирано следење third-party пакети

4. Unit тестирање

Unit тестовите валидираат индивидуални компоненти во изолација, обезбедувајќи секоја функција, метод и класа да се однесува правилно.

Детали за имплементација

Тестирање модели (tests/unit/test_models.py)

```
import ...

@pytest.mark.django_db  # smileska
class TestUserModel:

    def test_user_creation(self):  # smileska
        user = User.objects.create_user(
            username='testuser',
            email='test@example.com',
            password='testpass123'
        )
        assert user.username == 'testuser'
        assert user.email == 'test@example.com'
        assert user.check_password('testpass123')
        assert user.is_active
        assert not user.is_staff
        assert not user.is_superuser

    def test_profile_auto_creation(self):  # smileska
        user = User.objects.create_user(
            username='testuser',
            email='test@example.com',
            password='testpass123'
        )
        assert hasattr(user, 'profile')
        assert user.profile.status == 'Offline'
        assert not user.profile.is_verified
        assert user.profile.user == user
```

Клучни тест случаи:

- **User Model:** Создавање, валидација, автоматско генерирање профил
- **Message Model:** Валидација на содржина, управување на timestamp, релации
- **Chatroom Model:** Управување со членство, admin функционалност
- **Profile Model:** Ажурирање статус, verification workflows

Тестирање на интегритет на база:

- Foreign key ограничувања
- Валидација на уникатни полиња
- Cascade deletion однесување
- Прилагодени методи на модели

Метрики за покриеност

- **Покриеност на Model:** 95%+ линиска покриеност
- **Бизнис логика:** 90%+ branch покриеност
- **Гранични случаи:** Сеопфатно тестирање условите за грешки

5. Integration тестирање

Integration тестовите верификуваат комплетни кориснички работни процеси од автентикација до размена на пораки во реално време.

Тест сценарија

Authentication Flow (tests/integration/test_user_service_integration.py)

```
@pytest.mark.integration  # smileska
@pytest.mark.django_db
class TestAuthenticationFlow:

    def test_complete_registration_flow(self):  # smileska
        client = Client()

        response = client.get(reverse('accounts:register'))
        assert response.status_code == 200
        assert 'Create Account' in response.content.decode()

        registration_data = {
            'username': 'newuser',
            'email': 'newuser@example.com',
            'password1': 'TestPass123!',
            'password2': 'TestPass123!'
        }

        response = client.post(reverse('accounts:register'), registration_data)
        assert response.status_code == 302

        assert UnverifiedUser.objects.filter(username='newuser').exists()

        assert len(mail.outbox) == 1
        assert 'verify' in mail.outbox[0].subject.lower()
```

Комплетни работни процеси тестирани:

1. **Корисничка регистрација:** Поднесување на форма → email верификација → активација на сметка
2. **Најава/Одјава:** Автентикација → управување со сесија → ажурирање статус
3. **Приватни пораки:** Создавање на порака → доставување во реално време → управување со реакции
4. **Управување со соби:** Создавање → покана на корисници → admin контроли
5. **Upload датотеки:** Валидација на слика → складирање → генерирање URL

Cross-Component интеграција:

- Django ORM ↔ Redis cache
- WebSocket consumers ↔ Database модели
- Authentication middleware ↔ View дозволи
- Static file serving ↔ Media управување

Тестирање на комуникација во реално време

- **WebSocket врска:** Воспоставување и одржување врски
- **Message Broadcasting:** Доставување во реално време до повеќе клиенти
- **Отпорност на врска:** Управување прекини и повторни врски

6. API тестирање

Сеопфатно тестирање на REST API endpoints со фокус на валидација на податоци, управување со грешки и перформанси.

Стратегија за имплементација

Endpoint тестирање (tests/api/test_api.py)

```

@pytest.mark.django_db
class TestAPIComprehensive:

    def test_search_users_api_various_queries(self, multiple_users):
        client = Client()
        client.force_login(multiple_users[0])

        test_cases = [
            ('user', True),
            ('USER', True),
            ('nonexistent', False),
            ('us', True),
            ('a', False),
            ('user1', True),
            ('user123456', False),
        ]

        for query, should_find in test_cases:
            response = client.get(f'/chat/search-users/?q={query}')
            assert response.status_code == 200

            data = json.loads(response.content)
            assert 'users' in data
            assert isinstance(data['users'], list)

            if should_find and len(query) >= 2:
                usernames = [user['username'] for user in data['users']]
                assert multiple_users[0].username not in usernames

        print(f"Query '{query}': Found {len(data['users'])} users")

```

API тест категории:

1. User Search API

- **Валидација на барања:** проверка на минимална должина на внесениот текст, забрана или прифаќање на специјални знаци
- **Пагинација и ограничување на резултати:** проверка дали се применуваат лимитите за број на резултати по страница
- **Перформанси со големи сетови на податоци:** мерење на брзината и стабилноста при обработка на многу корисници

2. Message API

- CRUD операции
- Валидација на дозволи
- Управување со слики
- Управување со реакции

3. Chatroom API

- Управување со членови
- Валидација на admin привилегии
- Suggestion workflow
- Real-time ажурирања

Напредни API техники за тестирање:

- **Параметризирано тестирање:** Повеќе input сценарија
- **Response валидација:** JSON schema валидација
- **Управување со грешки:** HTTP status кодови и пораки за грешка
- **Rate Limiting:** Тестирање на заштита од злоупотреба на API

7. Безбедносно тестирање

Сеопфатно безбедносно тестирање што покрива вообичаени веб ранливости и Django-специфични безбедносни мерки.

Имплементација безбедносни тестови

Тестирање ранливости (tests/security/test_security.py)

```
@pytest.mark.django_db  # smileska
class TestSecuritySimple:

    def test_sql_injection_protection(self, user):  # smileska
        client = Client()
        client.force_login(user)

        malicious_query = "'; DROP TABLE auth_user; --"
        response = client.get(f'/chat/search-users/?q={malicious_query}')

        assert response.status_code == 200
        data = json.loads(response.content)
        assert 'users' in data

        users_exist = User.objects.all().exists()
        assert users_exist
```

Безбедносни области тестирани:

1. Injection напади

- SQL Injection превенција
- NoSQL injection (Redis)
- Заштита од Command injection

2. Cross-Site Scripting (XSS)

- Stored XSS во пораки
- Reflected XSS во search queries
- DOM-based XSS превенција

3. Authentication & Authorization

- Управување со сесии
- Барања за лозинки
- CSRF заштита
- Admin privilege escalation

4. Access Control

- Неавторизиран пристап до ресурси
- Валидација на членството во собата за разговор
- Ограничување на file upload

5. Input валидација

- Валидација на тип на датотека
- Content-type верификација
- Извршување на лимити на големина
- Управување со злонамерни filenames

Интеграција на безбедносни алатки:

- **Bandit:** Статичка анализа на код за безбедносни проблеми

- **Safety:** Скенирање на ранливости на зависности
- **Custom Security тестови:** Безбедносна валидација специфична за апликацијата

8. Тестирање на перформанси

Повеќеслојно тестирање на перформанси од индивидуални database queries до полно системско load тестирање.

Категории на тестови за перформанси

Database перформанси (tests/performance/test_performance.py)

```
@pytest.mark.performance  # smileska
@pytest.mark.django_db
class TestDatabasePerformance:

    def test_user_creation_performance(self):  # smileska
        start_time = time.time()

        users = []
        for i in range(20):
            user = User(
                username=f'user_{i}',
                email=f'user_{i}@example.com',
                password='pbkdf2_sha256$260000$fixed_hash_for_testing'
            )
            users.append(user)

        User.objects.bulk_create(users)

        end_time = time.time()
        creation_time = end_time - start_time

        print(f"Created 20 users in {creation_time:.3f} seconds")
        assert creation_time < 10.0
        assert User.objects.filter(username__startswith='user_').count() >= 20
```

Области за тестирање перформанси:

1. Database операции

- Валидација на оптимизација на queries
- Перформанси на Bulk операции
- Ефективност на индекси
- Ефикасност на connection pooling

2. View перформанси

- Времиња за вчитување страни
- API response времиња
- Template rendering перформанси
- Static file serving

3. Употреба на меморија

- Детекција на memory leak
- Управување со Bulk data
- Ефикасност на Cache
- Garbage collection мониторинг

4. Concurrency тестирање

- Симулација на повеќе корисници
- Database connection лимити
- Управување со WebSocket конекција
- Resource contention

Алатки за мониторинг перформанси:

- **psutil**: Мониторинг на системски ресурси
- **time модул**: Прецизно мерење на перформанси
- **Django Debug Toolbar**: Development анализа на перформанси
- **Custom метрики**: Следење на перформанси специфично за апликацијата

9. Frontend тестирање

Тестирање на компоненти на корисничкиот интерфејс, JavaScript функционалност и протоци на корисничка интеракција.

Имплементација Frontend тестови

UI Component тестирање (tests/frontend/test_ui_components.py)

```
class TestFrontendUIComponents(TestCase):
    # smileska

    def setUp(self):
        # smileska
        self.client = Client()
        self.user = User.objects.create_user(
            username='testuser',
            email='test@example.com',
            password='testpass123'
        )
        self.other_user = User.objects.create_user(
            username='otheruser',
            email='other@example.com',
            password='testpass123'
        )

    def test_login_page_ui_elements(self):
        # smileska
        response = self.client.get('/accounts/login/')
        self.assertEqual(response.status_code, 200)

        soup = BeautifulSoup(response.content, features='html.parser')

        self.assertIsNotNone(soup.find(name='form', attrs={'method': 'post'}))
        self.assertIsNotNone(soup.find(name='input', attrs={'name': 'username'}))
        self.assertIsNotNone(soup.find(name='input', attrs={'name': 'password'}))
```

Тестирање на JavaScript функционалност (tests/frontend/test_javascript_functionality.py)

```
class TestBasicFrontendFunctionality(TestCase):
    # smileska

    def setUp(self):
        # smileska
        self.client = Client()
        self.user = User.objects.create_user(
            username='testuser',
            email='test@example.com',
            password='testpass123'
        )
        self.other_user = User.objects.create_user(
            username='otheruser',
            email='other@example.com',
            password='testpass123'
        )

    def test_login_form_submission(self):
        # smileska
        response = self.client.post(path='/accounts/login/', data={
            'username': 'testuser',
            'password': 'testpass123'
        })
        self.assertEqual(response.status_code, 302)
```

Frontend тест категории:

1. Template Rendering

- Валидација на HTML структура
- Dynamic content rendering
- Template inheritance тестирање
- Достапност на Context variable

2. Form функционалност

- Form field валидација
- CSRF token присуство
- File upload интерфејси
- Error message приказ

3. JavaScript интеграција

- Воспоставување на WebSocket конекција
- Real-time message ажурирања
- User interaction управување
- Error управување и опоравување

4. Responsive дизајн

- Bootstrap class валидација
- Мобилна компатибилност
- CSS file вчитување
- Accessibility карактеристики

Frontend алатки за тестирање:

- **BeautifulSoup:** HTML parsing и валидација
- **Django Test Client:** Server-side rendering тестирање
- **Selenium WebDriver:** Browser автоматизација (кога е потребно)
- **Custom JavaScript тестирање:** Интеграција со Django views

10. Тестирање на оптоварување

Сеопфатно load тестирање користејќи Locust за симулирање real-world шеми и идентификување performance bottlenecks.

Имплементација Load тестирање

Locust тест сценарија (tests/performance/locustfile.py)

```

class ObscuraUser(HttpUser):  # smileska

    wait_time = between(min_wait: 1, max_wait: 3)

    def on_start(self):  # smileska
        self.username = f"user_{random.randint(a: 1000, b: 9999)}"
        self.email = f"{self.username}@example.com"
        self.password = "TestPass123!"

        self.register_user()
        self.login_user()

        self.chat_partners = []
        self.chatrooms = []

```

Load тест сценарија:

1. User симулациски класи

- **ObscuraUser:** Редовно корисничко однесување
- **AdminUser:** Административни акции
- **ChatroomHeavyUser:** Интензивна употреба на соби
- **APIPerformanceUser:** API-фокусирано тестирање

2. Тест сценарија

- Корисничка регистрација и автентикација
- Search функционалност под оптоварување
- Создавање и управување со соби
- Real-time messaging симулација
- File upload stress тестирање

3. Performance метрики

- Response time percentiles
- Throughput (requests/second)
- Error rates
- Resource utilization

- Concurrent user лимити

Бенефити на Load тестирање:

- **Идентификација на Bottleneck:** Database, Redis, WebSocket врски
- **Планирање на скалабилност:** Разбирање системски лимити
- **Регресија на перформанс:** Детекција на деградација на перформанси
- **Планирање на капацитет:** Препораки за Infrastructure sizing

11. CI/CD Pipeline за тестирање

Преглед

Автоматизиран testing pipeline со GitHub Actions за континуирана интеграција и испорака.

GitHub Actions конфигурација

CI/CD Workflow (.github/workflows/ci-cd.yml)

```
name: Obscura Messenger CI/CD
on:
  push:
    branches: [ master, main ]
  pull_request:
    branches: [ master, main ]
env:
  DOCKER_IMAGE: obscura-messenger
  REGISTRY: docker.io
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_PASSWORD: test_password
          POSTGRES_USER: test_user
          POSTGRES_DB: test_db
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432
```

Pipeline чекори:

1. Code Quality проверки

- Flake8 linting
- Bandit безбедносни скенирања
- Проверки на Safety dependency

2. Docker build и push

- Multi-platform image builds
- Автоматски tagging
- Registry push

3. Kubernetes deployment

- Deployment во test cluster
- Health check валидација
- Rollback capabilities

Бенефити на Pipeline:

- **Автоматизација:** Нема мануелни чекори
- **Брз feedback:** Брзо известување за проблеми
- **Quality gates:** Спречување на deployment со failing тестови
- **Транспарентност:** Јавни тест резултати и покриеност

12. Извршување и извештаи за тестови

Сеопфатен систем за извршување тестови и генерирање детални извештаи.

Test Runner систем

Прилагоден тест runner (run_tests.py)

```

class TestRunner:  # smileska
    def __init__(self):  # smileska
        self.project_root = Path.cwd()
        self.reports_dir = Path("tests/reports")
        self.reports_dir.mkdir(parents=True, exist_ok=True)
        self.start_time = None
        self.results = {}

    def clean_test_environment(self):  # 3 usages  # smileska
        print("Cleaning test environment...")

        test_db_files = [
            'test_db.sqlite3',
            'tests/test_db.sqlite3',
            'db.sqlite3',
            '.pytest_cache',
        ]

        for item in test_db_files:
            path = Path(item)
            if path.exists():
                try:
                    if path.is_dir():
                        import shutil
                        shutil.rmtree(path)
                        print(f"Removed directory {item}")
                    else:

```

Автоматизирани извештаи

```

def generate_report(self):  # 2 usages  # smileska
    report_file = self.reports_dir / f"test_report_{datetime.now().strftime('%Y%m%d_%H%M%S')}.json"

    total_duration = sum(r.get('duration', 0) for r in self.results.values())
    successful_tests = sum(1 for r in self.results.values() if r.get('success', False))
    total_tests = len(self.results)

    report = {
        'timestamp': datetime.now().isoformat(),
        'summary': {
            'total_tests': total_tests,
            'successful_tests': successful_tests,
            'failed_tests': total_tests - successful_tests,
            'success_rate': (successful_tests / total_tests * 100) if total_tests > 0 else 0,
            'total_duration': total_duration
        },
        'results': self.results
    }

    with open(report_file, 'w') as f:
        json.dump(report, f, indent=2)

    print(f"\nTest Report Generated: {report_file}")
    return report

```

13. Заклучок

Овој проект демонстрира **сеопфатен пристап кон тестирање на софтвер** што ги надминува основните техники покриени во курсот. Клучните постигнувања вклучуваат:

Технички иновации

1. Скалабилна архитектура

- Микро-сервисна testing архитектура
- Containerized тест околина
- Kubernetes deployment тестирање
- Multi-environment validation

2. Автоматизација и CI/CD

- GitHub Actions pipeline интеграција
- Автоматизирани безбедносни скенирања
- Детекција на Performance regression
- Автоматизирани reporting и alerting

Покриеност на тестирање

Obscura Messenger тестирањето демонстрира **добри testing practices** што обезбедуваат високо ниво на софтверски квалитет, сигурност и одржливост. Овој проект служи како **comprehensive reference** за модерно тестирање на методологии и може да биде користен како template за слични апликации.

Комбинацијата на традиционални тестирачки техники со cutting-edge пристапи создава **цврста основа** за целиот процес на развој на софтвер што обезбедува долгорочен успех и скалабилност.